UNIVERSIDAD POLITÉCNICA
BICENTENARIO

Controlador ON-OFF con Keypad

Presenta:

- **Ailton Alejandro Trujillo Lopez. 24033482**
- **Edgar Ivan Gutierrez Gutierrez. 24033101**
- **Ruben Antonio Arredondo Villegas. 24032957**
- **Roberto Leonel Mendoza Nuñez. 24033370**

Asignatura:

SISTEMAS EMBEBIDOS

Profesor:

Hugo Antonio Méndez Guzmán

Silao, Gto. a 15 de octubre de 2026

Contenido

Introducción 3

Marco Teórico 3

Desarrollo 5

Resultados 14

Observaciones y Conclusiones 14

Referencias 16

Lista de Figuras

Figura 1. Circuito electrónico del sistema del horno 6

Figura 2. Montaje final del sistema del horno 14

Introducción

En esta práctica se realizó un sistema de control automático de temperatura utilizando una tarjeta ESP32, un sensor LM35 y un teclado matricial 4×4. El objetivo principal fue desarrollar un dispositivo capaz de medir la temperatura del ambiente y controlar automáticamente un sistema de calentamiento y enfriamiento.

El funcionamiento del proyecto consiste en utilizar el sensor LM35 para obtener la temperatura actual, mientras que mediante el teclado matricial se pueden establecer los límites máximos y mínimos de temperatura. Estos valores se muestran en una pantalla LCD de 16×2, permitiendo visualizar las mediciones y el estado del sistema.

Para controlar la temperatura se utilizaron relevadores que permiten activar un foco cuando se necesita aumentar la temperatura y una celda Peltier acompañada de ventiladores cuando se necesita disminuirla. De esta manera, el sistema puede alternar entre calentamiento y enfriamiento de acuerdo con los valores establecidos.

Esta práctica permitió poner en práctica conocimientos de programación, electrónica y automatización, además de comprender cómo se pueden integrar diferentes componentes electrónicos para desarrollar un sistema de control.

Marco Teórico

ESP32

El ESP32 es una tarjeta de desarrollo que cuenta con un microcontrolador capaz de ejecutar instrucciones programadas y controlar diferentes dispositivos electrónicos. Dispone de entradas y salidas digitales y analógicas, lo que permite conectarle sensores, pantallas, teclados y módulos de relevadores. En esta práctica se utiliza como unidad principal para procesar la información del sensor y controlar los sistemas de calentamiento y enfriamiento.

Sensor de temperatura LM35

El LM35 es un sensor electrónico que permite medir la temperatura y convertirla en una señal de voltaje. Su principal característica es que proporciona una salida proporcional a la temperatura, con una relación aproximada de 10 milivoltios por cada grado Celsius. En el proyecto se utiliza para medir la temperatura y enviar la información al ESP32.

Teclado matricial 4×4

Es un dispositivo de entrada formado por 16 teclas distribuidas en cuatro filas y cuatro columnas. Permite introducir números y comandos mediante una conexión con el microcontrolador. En esta práctica se utiliza para configurar los valores máximos y mínimos de temperatura que deberá mantener el sistema.

Pantalla LCD 16×2 con módulo I2C

La pantalla LCD es un dispositivo de visualización que permite mostrar información mediante dos filas de 16 caracteres cada una. El módulo I2C facilita su conexión con el ESP32, ya que utiliza principalmente dos líneas de comunicación: SDA y SCL. Se utiliza para mostrar la temperatura actual, los límites establecidos y el estado del sistema.

Módulo de relevadores

Los relevadores son dispositivos electromecánicos que funcionan como interruptores controlados mediante señales eléctricas. Permiten controlar cargas externas utilizando una señal de bajo voltaje proveniente del microcontrolador. En este proyecto se utilizan para activar y desactivar el foco, la celda Peltier y los ventiladores.

Celda Peltier

La celda Peltier es un componente electrónico que permite transferir calor de una de sus caras hacia la otra cuando recibe corriente eléctrica. Esto provoca que una cara se enfríe mientras la otra se calienta. En esta práctica se utiliza como parte del sistema de enfriamiento, acompañada de un ventilador para ayudar a disipar el calor.

Ventiladores de 12 V

Los ventiladores son dispositivos que utilizan energía eléctrica para generar un flujo de aire. Su función es ayudar a disipar el calor producido por la celda Peltier y facilitar la circulación del aire durante el proceso de enfriamiento.

Sistema de control ON-OFF

Es un método de control automático que trabaja mediante dos estados principales: encendido y apagado. En este proyecto, cuando la temperatura alcanza el límite máximo establecido, se activa el sistema de enfriamiento; cuando desciende hasta el límite mínimo, se activa nuevamente el calentamiento. Este proceso se repite para mantener la temperatura dentro del rango seleccionado.

Desarrollo

Material requerido

- 1 ESP32 DevKit V1
- 1 Sensor de temperatura LM35
- 1 Teclado matricial 4×4
- 1 Pantalla LCD 16×2 con módulo I2C
- 1 Módulo de relevadores de 4 canales (5 V)
- 1 Celda Peltier TEC1-12706
- 2 Ventiladores de 12 V
- 1 Foco de 120 V CA
- 1 Fuente de alimentación de 12 V
- 1 Protoboard
- Varios Cables jumper macho-macho
- Varios Cables jumper macho-hembra
- 1 Cable USB para ESP32
- 1 Computadora con Arduino IDE

Como primera parte se desarrolló …

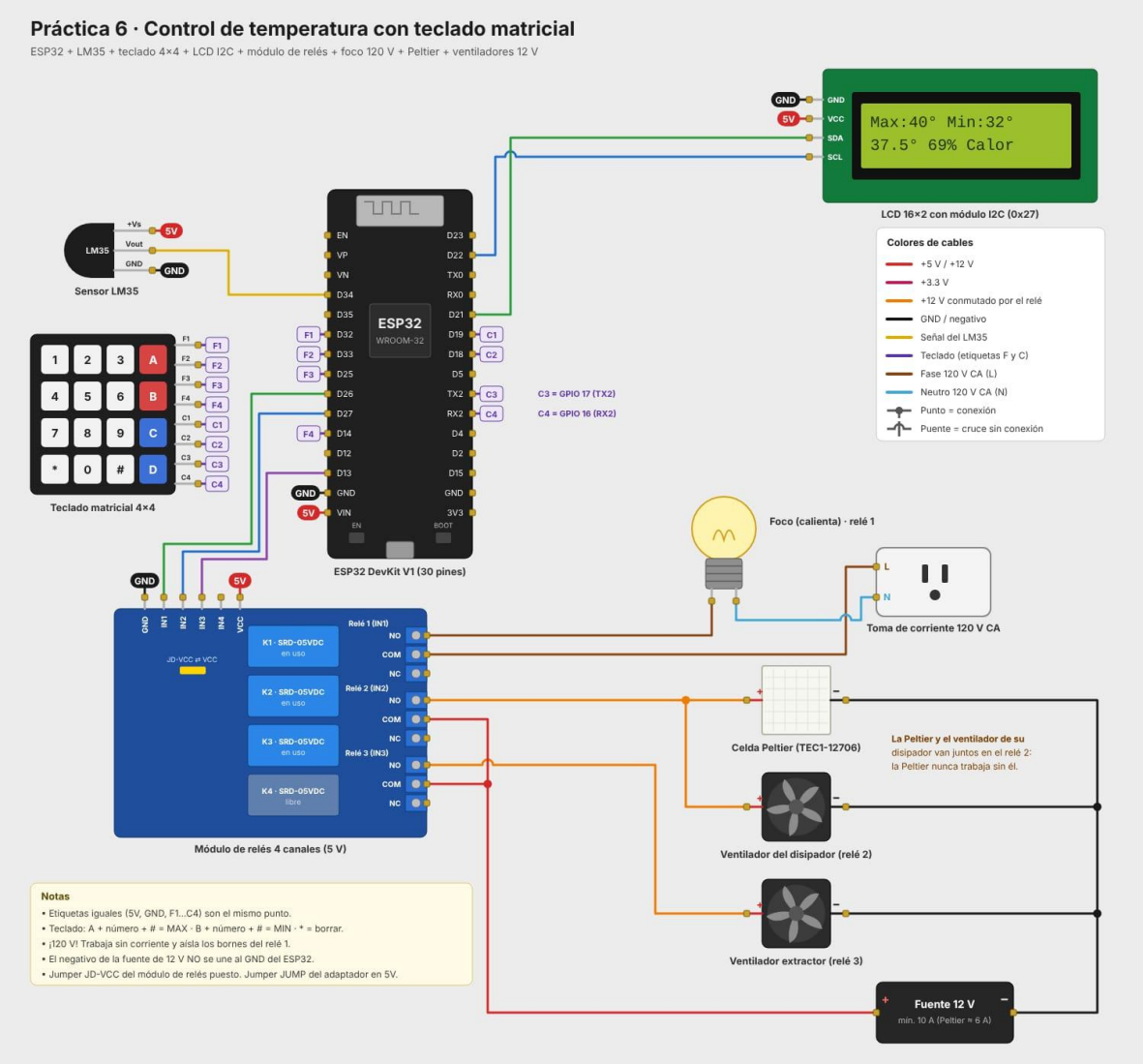


Figura 1. Circuito electrónico del sistema del horno

Enseguida se procedió a …

A continuación, se muestra el código empleado.

Código utilizado en el IDLE de Arduino
// A + numero + # = MAX | B + numero + # = MIN | * = borrar #include <Wire.h> #include <LiquidCrystal_I2C.h> #include <Keypad.h> #include <Preferences.h> // ---------- CONFIGURACION DE PINES ---------- #define PIN_LM35 34 // salida (VOUT) del LM35, alimentado a 5V #define RELAY_FOCO 26 // IN4 - foco (CALIENTA)

```
#define RELAY_PELTIER     27   // IN1 - celda Peltier + su ventilador (ENFRIA)
#define RELAY_EXTRACTOR   13   // IN2 - ventilador extractor (solo al ENFRIAR)

#define LCD_SDA          21
#define LCD_SCL          22

// La mayoria de modulos de rele son "activos en BAJO" (LOW).
// Si tus reles funcionan al reves, cambia LOW por HIGH.
#define RELAY_ACTIVO   LOW
#define RELAY_INACTIVO (!RELAY_ACTIVO)

// ---------------------- TECLADO 4x4 ----------------------
const byte FILAS = 4;
const byte COLUMNAS = 4;
char teclas[FILAS][COLUMNAS] = {
  { '1', '2', '3', 'A' },
  { '4', '5', '6', 'B' },
  { '7', '8', '9', 'C' },
  { '*', '0', '#', 'D' }
};
byte pinesFilas[FILAS]      = { 32, 33, 25, 14 };  // F1, F2, F3, F4
byte pinesColumnas[COLUMNAS] = { 19, 18, 17, 16 };  // C1, C2, C3, C4
Keypad teclado = Keypad(makeKeymap(teclas), pinesFilas, pinesColumnas, FILAS, COLUMNAS);

// ---------------------- PARAMETROS DE CONTROL ----------------------
const int TEMP_MAX_INICIAL = 60;  // C - valor de fabrica del MAX
const int TEMP_MIN_INICIAL = 30;  // C - valor de fabrica del MIN
const int LIMITE_INFERIOR  = 0;   // C - lo mas bajo que se puede escribir
const int LIMITE_SUPERIOR  = 99;  // C - lo mas alto que se puede escribir
const int MARGEN_MINIMO    = 5;   // separacion minima entre MAX y MIN (grados)

const unsigned long PAUSA_CAMBIO_MS = 1000;     // foco y Peltier apagados al cambiar
const int MAX_ERRORES_SENSOR = 3;               // lecturas fallidas seguidas antes de parar
const unsigned long TIEMPO_CAPTURA_MS = 10000;  // cancela la captura si no se teclea nada
const unsigned long TIEMPO_MENSAJE_MS = 2000;   // tiempo que se muestra un aviso

// ---------------------- OBJETOS GLOBALES ----------------------
LiquidCrystal_I2C lcd(0x27, 16, 2);  // si no se ve nada, prueba 0x3F
Preferences prefs;

// ---------------------- ESTADO DEL SISTEMA ----------------------
enum EstadoCiclo { INICIANDO, CALENTANDO, ENFRIANDO, ERROR_SENSOR };
EstadoCiclo estado = INICIANDO;

int tempMax = TEMP_MAX_INICIAL;
int tempMin = TEMP_MIN_INICIAL;

float tempActual   = 0.0;
float porcentaje   = 0.0;
int   erroresSensor = 0;
bool  hayLectura   = false;

unsigned long momentoCambioEstado = 0;

// Lectura del LM35 cada 100 ms con promedio movil de 5 muestras
unsigned long momentoLectura = 0;
const unsigned long INTERVALO_SENSOR_MS = 100;
const int MUESTRAS_FILTRO = 5;
float muestras[MUESTRAS_FILTRO];
int indiceMuestra = 0;
bool filtroListo = false;
const int MV_MAXIMO_VALIDO = 1500;  // mas de 1.5 V (150 C) = sensor mal conectado

// Captura desde el teclado
enum Pantalla { NORMAL, CAPTURANDO, MENSAJE };
Pantalla pantalla = NORMAL;
```

```
bool capturandoMax = true;        // true = escribiendo MAX, false = MIN
char digitos[3] = "";             // hasta 2 digitos (0-99)
unsigned long ultimaTecla = 0;
char mensaje1[17] = "";
char mensaje2[17] = "";
unsigned long momentoMensaje = 0;

unsigned long ultimaActualizacionLCD = 0;
const unsigned long INTERVALO_LCD_MS = 300;

const char GRADO = (char)223;  // simbolo de grados en la LCD

// ---------------------- DECLARACION DE FUNCIONES ----------------------
void manejarTeclado();
void iniciarCaptura(bool esMax);
void confirmarCaptura();
void mostrarMensaje(const char* l1, const char* l2);
void validarLimites();
void guardarConfiguracion();
void leerSensor();
void actualizarCiclo();
void cambiarEstado(EstadoCiclo nuevo);
void aplicarSalidas();
void apagarFocoYPeltier();
void actualizarExtractor();
const char* nombreEstado();
void actualizarLCD();
void imprimirLinea(int fila, const char* texto);

// ========================================================================
void setup() {
  Serial.begin(115200);

  pinMode(RELAY_FOCO, OUTPUT);
  pinMode(RELAY_PELTIER, OUTPUT);
  pinMode(RELAY_EXTRACTOR, OUTPUT);
  apagarFocoYPeltier();
  digitalWrite(RELAY_EXTRACTOR, RELAY_INACTIVO);  // el extractor arranca apagado

  teclado.setDebounceTime(50);

  // LCD
  delay(500);
  Wire.begin(LCD_SDA, LCD_SCL);
  Wire.setClock(100000);
  lcd.init();
  lcd.backlight();
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Iniciando...");

  // LM35: entrada analogica de 0 a 3.3 V
  analogReadResolution(12);
  analogSetPinAttenuation(PIN_LM35, ADC_11db);
  pinMode(PIN_LM35, INPUT);

  // Cargar MAX y MIN guardados (si no hay nada guardado: 60 y 30)
  prefs.begin("tempTeclado", false);
  tempMax = prefs.getInt("tempMax", TEMP_MAX_INICIAL);
  tempMin = prefs.getInt("tempMin", TEMP_MIN_INICIAL);
  prefs.end();
  validarLimites();

  Serial.println("=== Control de temperatura con teclado ===");
  Serial.println("A + numero + # = MAX | B + numero + # = MIN | * = borrar");
  Serial.print("MAX: "); Serial.print(tempMax); Serial.print(" C  MIN: "); Serial.print(tempMin); Serial.println(" C");
```

```
  delay(1000);
  lcd.clear();
}

// ====================================================================
void loop() {
  manejarTeclado();

  // Leer el LM35 y actualizar el ciclo cada 100 ms
  if (millis() - momentoLectura >= INTERVALO_SENSOR_MS) {
    momentoLectura = millis();
    leerSensor();
    actualizarCiclo();
  }

  if (millis() - ultimaActualizacionLCD >= INTERVALO_LCD_MS) {
    ultimaActualizacionLCD = millis();
    actualizarLCD();
  }
}

// ====================================================================
// TECLADO
void manejarTeclado() {
  // Cancelar la captura si se dejo a medias
  if (pantalla == CAPTURANDO && millis() - ultimaTecla > TIEMPO_CAPTURA_MS) {
    mostrarMensaje("Captura", "cancelada");
  }
  // Regresar a la pantalla normal despues de un aviso
  if (pantalla == MENSAJE && millis() - momentoMensaje > TIEMPO_MENSAJE_MS) {
    pantalla = NORMAL;
  }

  char tecla = teclado.getKey();
  if (!tecla) return;

  ultimaTecla = millis();
  Serial.print("Tecla: "); Serial.println(tecla);

  if (tecla == 'A') { iniciarCaptura(true);  return; }
  if (tecla == 'B') { iniciarCaptura(false); return; }

  if (pantalla != CAPTURANDO) return;  // fuera de captura solo importan A y B

  int n = strlen(digitos);
  if (tecla >= '0' && tecla <= '9') {
    if (n < 2) {
      digitos[n] = tecla;
      digitos[n + 1] = '\0';
    }
  } else if (tecla == '*') {
    if (n > 0) digitos[n - 1] = '\0';         // borra el ultimo digito
    else mostrarMensaje("Captura", "cancelada");  // sin digitos: cancela
  } else if (tecla == '#') {
    confirmarCaptura();
  }
  // C y D no tienen funcion
}

void iniciarCaptura(bool esMax) {
  capturandoMax = esMax;
  digitos[0] = '\0';
  pantalla = CAPTURANDO;
  ultimaTecla = millis();
}
```

```
void confirmarCaptura() {
  if (strlen(digitos) == 0) {
    mostrarMensaje("Escribe un", "numero primero");
    return;
  }

  int valor = atoi(digitos);
  int nuevoMax = capturandoMax ? valor : tempMax;
  int nuevoMin = capturandoMax ? tempMin : valor;

  if (valor < LIMITE_INFERIOR || valor > LIMITE_SUPERIOR) {
    mostrarMensaje("Fuera de rango", "Usa 0 a 99");
    return;
  }
  if (nuevoMax < nuevoMin + MARGEN_MINIMO) {
    char l2[24];
    if (capturandoMax) snprintf(l2, sizeof(l2), "MAX >= %d%c", tempMin + MARGEN_MINIMO, GRADO);
    else               snprintf(l2, sizeof(l2), "MIN <= %d%c", tempMax - MARGEN_MINIMO, GRADO);
    mostrarMensaje("Valor no valido", l2);
    return;
  }

  tempMax = nuevoMax;
  tempMin = nuevoMin;
  guardarConfiguracion();

  char l1[24];
  snprintf(l1, sizeof(l1), "%s = %d%c", capturandoMax ? "MAX" : "MIN", valor, GRADO);
  mostrarMensaje(l1, "Guardado");
}

void mostrarMensaje(const char* l1, const char* l2) {
  strncpy(mensaje1, l1, 16); mensaje1[16] = '\0';
  strncpy(mensaje2, l2, 16); mensaje2[16] = '\0';
  pantalla = MENSAJE;
  momentoMensaje = millis();
  Serial.print("LCD: "); Serial.print(mensaje1); Serial.print(" / "); Serial.println(mensaje2);
}

// Asegura valores validos (por si en memoria habia algo raro)
void validarLimites() {
  tempMax = constrain(tempMax, LIMITE_INFERIOR + MARGEN_MINIMO, LIMITE_SUPERIOR);
  tempMin = constrain(tempMin, LIMITE_INFERIOR, tempMax - MARGEN_MINIMO);
}

void guardarConfiguracion() {
  prefs.begin("tempTeclado", false);
  prefs.putInt("tempMax", tempMax);
  prefs.putInt("tempMin", tempMin);
  prefs.end();
  Serial.print("MAX: "); Serial.print(tempMax); Serial.print(" C  MIN: "); Serial.print(tempMin); Serial.println(" C");
}

// ================================================================
// SENSOR LM35
void leerSensor() {
  int milivoltios = analogReadMilliVolts(PIN_LM35);

  // El LM35 entrega 10 mV por grado: mas de 1.5 V indica una falla de conexion
  if (milivoltios > MV_MAXIMO_VALIDO) {
    erroresSensor++;
    filtroListo = false;
    Serial.println("Error: revisar LM35");
    return;
  }
```

```
  float lectura = milivoltios / 10.0;

  // Promedio movil de 5 lecturas (la primera llena el filtro completo
  // para no promediar contra ceros)
  if (!filtroListo) {
    for (int i = 0; i < MUESTRAS_FILTRO; i++) muestras[i] = lectura;
    filtroListo = true;
  } else {
    muestras[indiceMuestra] = lectura;
  }
  indiceMuestra = (indiceMuestra + 1) % MUESTRAS_FILTRO;
  float suma = 0;
  for (int i = 0; i < MUESTRAS_FILTRO; i++) suma += muestras[i];
  float t = suma / MUESTRAS_FILTRO;

  erroresSensor = 0;
  hayLectura = true;
  tempActual = t;

  // % = posicion de la temperatura entre MIN (0%) y MAX (100%)
  porcentaje = (tempActual - tempMin) * 100.0 / (tempMax - tempMin);
  porcentaje = constrain(porcentaje, 0.0, 100.0);

  Serial.print("LM35: ");       Serial.print(lectura, 1);
  Serial.print(" C | Filtrada: "); Serial.print(tempActual, 1);
  Serial.print(" C | ");        Serial.print(porcentaje, 0);
  Serial.print(" % | Estado: "); Serial.println(nombreEstado());
}

// ======================================================================
// CICLO: CALENTANDO hasta MAX -> ENFRIANDO hasta MIN -> CALENTANDO ...
void actualizarCiclo() {
  // Seguridad: si el sensor falla varias veces seguidas, apagar foco y Peltier
  if (erroresSensor >= MAX_ERRORES_SENSOR) {
    if (estado != ERROR_SENSOR) cambiarEstado(ERROR_SENSOR);
    apagarFocoYPeltier();
    actualizarExtractor();
    return;
  }

  if (!hayLectura) return;

  switch (estado) {
    case INICIANDO:
    case ERROR_SENSOR:
      if (erroresSensor == 0) {
        cambiarEstado(tempActual >= tempMax ? ENFRIANDO : CALENTANDO);
      }
      break;

    case CALENTANDO:
      if (tempActual >= tempMax) cambiarEstado(ENFRIANDO);
      break;

    case ENFRIANDO:
      if (tempActual <= tempMin) cambiarEstado(CALENTANDO);
      break;
  }

  aplicarSalidas();
}

void cambiarEstado(EstadoCiclo nuevo) {
  estado = nuevo;
  momentoCambioEstado = millis();
```

```
  apagarFocoYPeltier();   // los dos apagados durante la pausa de cambio
  actualizarExtractor();
  Serial.print(">>> Cambio de estado: "); Serial.println(nombreEstado());
}

// Extractor: encendido solo al ENFRIAR (y en error del sensor, para sacar
// el calor que quede). Apagado al CALENTAR para no perder el calor del foco.
void actualizarExtractor() {
  bool encender = (estado == ENFRIANDO || estado == ERROR_SENSOR);
  digitalWrite(RELAY_EXTRACTOR, encender ? RELAY_ACTIVO : RELAY_INACTIVO);
}

void aplicarSalidas() {
  actualizarExtractor();

  if (millis() - momentoCambioEstado < PAUSA_CAMBIO_MS) {
    apagarFocoYPeltier();
    return;
  }

  if (estado == CALENTANDO) {
    digitalWrite(RELAY_PELTIER, RELAY_INACTIVO);  // primero apagar el otro
    digitalWrite(RELAY_FOCO,    RELAY_ACTIVO);
  } else if (estado == ENFRIANDO) {
    digitalWrite(RELAY_FOCO,    RELAY_INACTIVO);
    digitalWrite(RELAY_PELTIER, RELAY_ACTIVO);
  } else {
    apagarFocoYPeltier();
  }
}

void apagarFocoYPeltier() {
  digitalWrite(RELAY_FOCO,    RELAY_INACTIVO);
  digitalWrite(RELAY_PELTIER, RELAY_INACTIVO);
}

const char* nombreEstado() {
  switch (estado) {
    case CALENTANDO:   return "CALENTANDO";
    case ENFRIANDO:    return "ENFRIANDO";
    case ERROR_SENSOR: return "ERROR SENSOR";
    default:           return "INICIANDO";
  }
}

// ========================================================================
// LCD 16x2
void actualizarLCD() {
  char linea[17];

  if (pantalla == CAPTURANDO) {
    // "Nuevo MAX: 45_" con un guion bajo donde va el siguiente digito
    snprintf(linea, sizeof(linea), "Nuevo %s: %s%s",
          capturandoMax ? "MAX" : "MIN", digitos, strlen(digitos) < 2 ? "_" : "");
    imprimirLinea(0, linea);
    imprimirLinea(1, "#:OK  *:Borrar");
    return;
  }

  if (pantalla == MENSAJE) {
    imprimirLinea(0, mensaje1);
    imprimirLinea(1, mensaje2);
    return;
  }

  // Pantalla normal
```

```
  snprintf(linea, sizeof(linea), "Max:%2d%c Min:%2d%c", tempMax, GRADO, tempMin, GRADO);
  imprimirLinea(0, linea);

  if (estado == ERROR_SENSOR) {
    imprimirLinea(1, "Error LM35");
  } else if (!hayLectura) {
    imprimirLinea(1, "Leyendo LM35");
  } else {
    const char* textoEstado;
    if (estado == CALENTANDO)     textoEstado = "Calor";
    else if (estado == ENFRIANDO) textoEstado = "Frio";
    else                          textoEstado = "";

    snprintf(linea, sizeof(linea), "%4.1f%c %3d%% %s",
           tempActual, GRADO, (int)(porcentaje + 0.5), textoEstado);
    imprimirLinea(1, linea);
  }
}

// Escribe un texto en la fila indicada y rellena con espacios hasta 16
// caracteres, para borrar restos de textos anteriores sin parpadeo
void imprimirLinea(int fila, const char* texto) {
  lcd.setCursor(0, fila);
  int n = 0;
  while (texto[n] != '\0' && n < 16) {
    lcd.print(texto[n]);
    n++;
  }
  while (n < 16) {
    lcd.print(' ');
    n++;
  }
}
```

Resultados

.

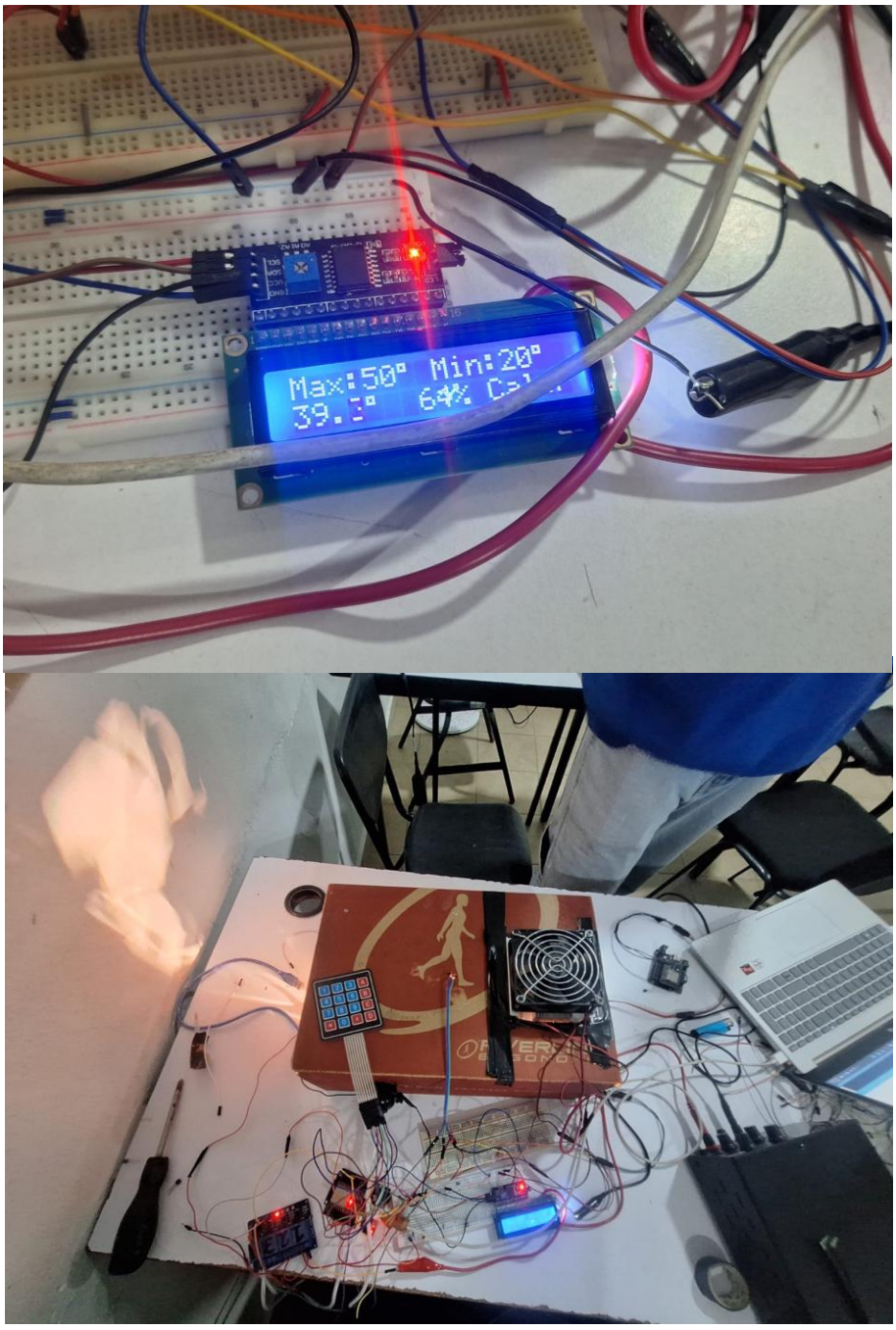


Figura 2. Montaje final del sistema del horno

Observaciones y Conclusiones

Observaciones

Durante el desarrollo de la práctica se identificó la importancia de conectar correctamente cada uno de los componentes electrónicos para lograr el funcionamiento adecuado del sistema. Se observó que el sensor LM35 permite obtener lecturas de temperatura que son procesadas por el ESP32 y posteriormente mostradas en la pantalla LCD.

También se comprobó la utilidad del teclado matricial para establecer los valores máximos y mínimos de temperatura sin necesidad de modificar directamente el código de programación. Los relevadores permiten controlar los dispositivos encargados de calentar y enfriar, de acuerdo con las condiciones establecidas.

Es importante considerar que el sistema utiliza una pequeña pausa al cambiar entre calentamiento y enfriamiento para evitar que ambos dispositivos principales funcionen al mismo tiempo. Además, el programa contempla una condición de seguridad para detener el calentamiento y el enfriamiento cuando se detecta una falla en el sensor.

Conclusiones

Con la realización de esta práctica se logró comprender cómo funciona un sistema automático de control de temperatura mediante la integración de diferentes componentes electrónicos y programación.

Se aprendió a utilizar el ESP32 como unidad principal de control, así como a implementar un sensor de temperatura, una pantalla LCD y un teclado matricial para ingresar y visualizar información. También se comprendió la función de los relevadores para controlar dispositivos externos, como el foco y la celda Peltier.

En conclusión, esta práctica permitió reforzar los conocimientos de electrónica, programación y automatización, demostrando cómo es posible construir un sistema capaz de regular la temperatura de manera automática mediante valores previamente establecidos.

Referencias

[1] Texas Instruments. (2017). LM35 Precision Centigrade Temperature Sensors. Datasheet. Texas Instruments.
Consultar hoja de datos del sensor LM35.

[2] Espressif Systems. ESP32 Series Datasheet. Documentación técnica del microcontrolador ESP32.
Documentación oficial de ESP32.

[3] Arduino. Documentación oficial de Arduino. Referencia para programación y uso del entorno Arduino IDE.
Arduino Documentation.

[4] Código fuente y diagrama de conexiones elaborados para la práctica 6: Control de temperatura con teclado matricial.